NESBA

National Entrepreneurship & Small Business Academy

Platform System Design

How the NESBA learning platform works: the apps people use (frontend), the services behind them (backend), and the flows that connect them.

VERSION	PLATFORMS	STACK
October 2026	Web · iPhone · Android	Expo / React Native · Supabase · LiveKit · Stripe

Contents

1 The platform at a glance

2 Technology stack

3 Frontend: app structure and navigation

4 Frontend: screens and shared building blocks

5 Backend: Supabase services

6 Backend: server functions and scheduled jobs

7 Data model

8 Users, roles and permissions

9 Flow: sign-up, login and onboarding

10 Flow: posts, stories and uploads

11 Flow: courses, lessons, quizzes and certificates

12 Flow: buying a premium course

13 Flow: booking a private session

14 Flow: live sessions and recordings

15 Flow: notifications and push

16 Flow: practical projects

17 Security model

18 Configuration, deployment and operations

19 Capacity and costs (year 1)

20 Go-live checklist

> **How to read this document.** Boxes and arrows show which part talks to which. “Frontend” is everything that runs on the user’s phone or in their browser. “Backend” is everything that runs on servers NESBA rents (Supabase, LiveKit, Stripe). NESBA runs no servers of its own: every backend piece is a managed service.

1 · The platform at a glance

One codebase builds three apps (website, iPhone, Android). All of them talk to the same backend.

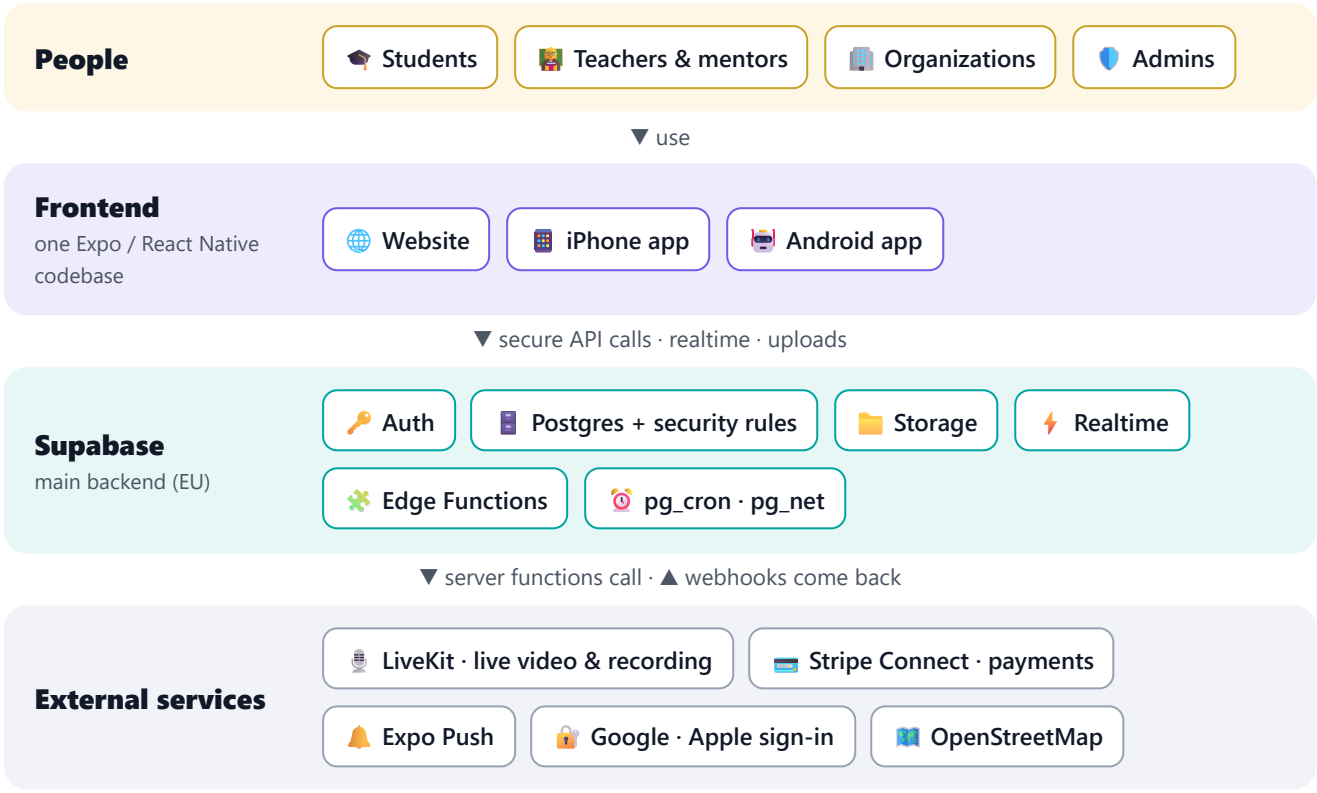


Figure 1 · System overview. The apps connect directly to LiveKit for live audio and video, after a server function has issued a token.

Learn

Courses with sections and lessons, videos and files, quizzes, assignments, progress, certificates and badges.

Connect

Social feed, stories, comments, follows, direct messages, organizations and practical team projects.

Live & earn

Live classes in a 3D lecture hall, private 1-to-1 sessions, recordings, paid courses and creator payouts.

2 · Technology stack

Layer	Technology	What it does for NESBA
App framework	Expo SDK 54 · React Native 0.81 · React 19	One JavaScript codebase that becomes the website, the iPhone app and the Android app.
Navigation	Expo Router 6	File-based screens: every file in `src/app` is a page with its own web address (e.g. `/course/123`).
Web rendering	react-native-web	Turns the phone components into a normal website.
Database	Supabase Postgres 17	All data: profiles, courses, posts, messages, payments, projects. Security rules live in the database itself.
Accounts	Supabase Auth	Email/password, Google and Apple sign-in, password reset, sessions.
Files	Supabase Storage	Avatars, post and story media, lesson files, message attachments, recordings.
Live updates	Supabase Realtime	New messages, notifications and live-room changes appear instantly.
Server code	Supabase Edge Functions (Deno)	Payments, live-room tokens and moderation, recording, push delivery, clean-up.
Schedules	pg_cron + pg_net	Nightly story clean-up; database triggers that call server functions.
Live sessions	LiveKit Cloud + livekit-client	Real-time audio, camera, screen sharing, reactions and recording (egress).
3D lecture hall	three.js · React Three Fiber · drei	The 3D room shown during live sessions (loaded only when a room opens).
Payments	Stripe Connect (Express) · Checkout	Course sales and private sessions; 80% to the creator, 20% platform fee.
Phone features	expo-notifications · expo-calendar · expo-location · expo-image-picker · expo-av · expo-sqlite	Push notifications, add-to-calendar, story location, picking media, audio/video playback, saved theme.
Developer tools	Playwright · Supabase CLI · ESLint · Context7 / Playwright MCP	Automatic screenshots and error checks, deploying functions, code quality, up-to-date docs.

> All purchases happen on the **website**. The phone apps open the website to buy, so Apple and Google take no in-app purchase commission.

3 · Frontend: app structure and navigation

Five tabs at the bottom; every other page opens on top of them.

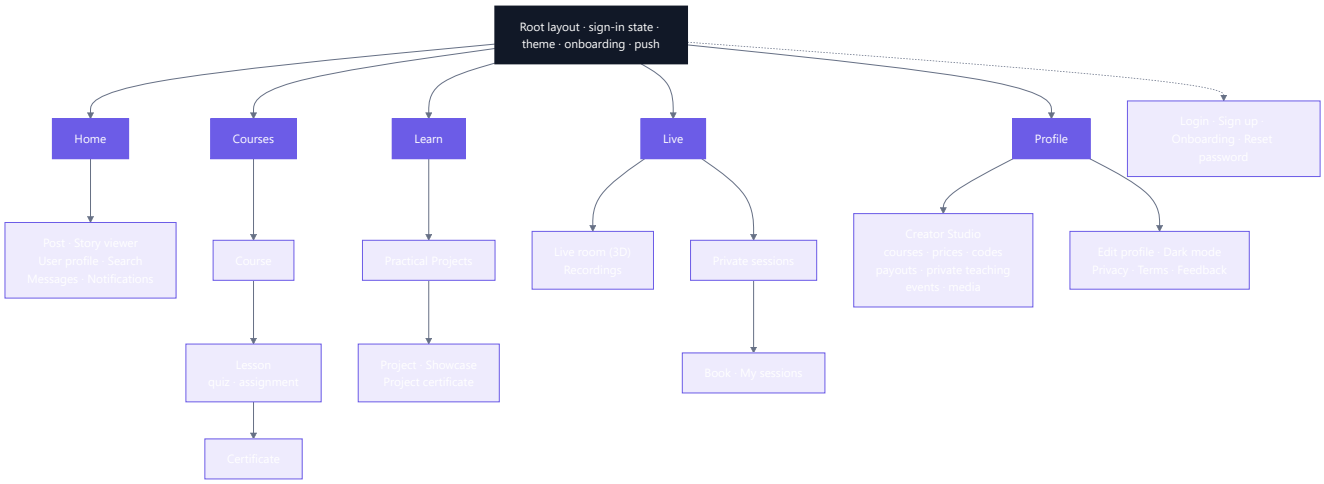


Figure 2 · Navigation map (simplified). Every page also has its own web address, so links can be shared.

How a screen gets its data

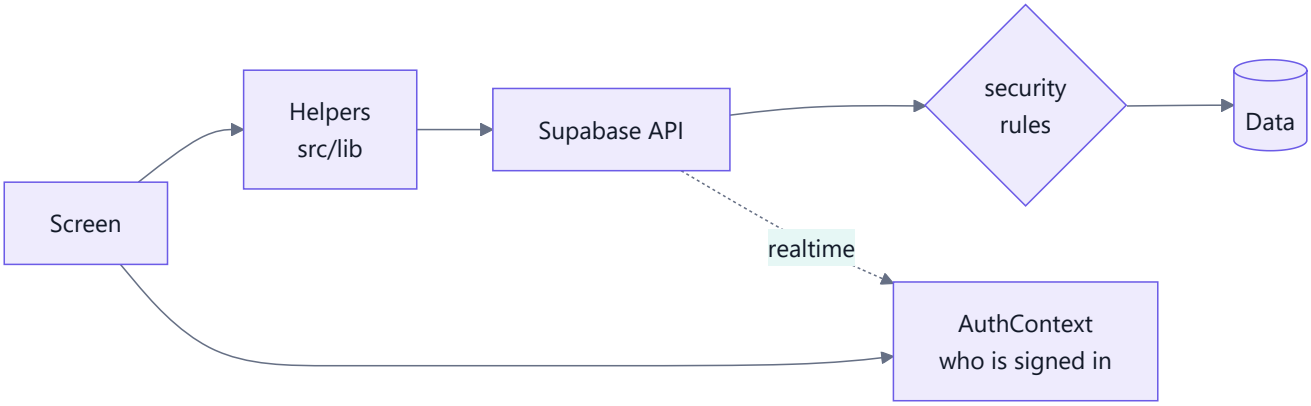


Figure 3 · Screens never talk to the database directly; they use small helper modules.

4 · Frontend: screens and shared building blocks

Area	Screens (file paths under `src/app`)
Tabs	`(tabs)/index` Home · `courses` · `learn` · `live` · `profile`
Accounts	`login` · `signup/` (student, teacher, organization) · `onboarding` · `reset-password` · `auth/callback` · `edit-profile` · `profile/edit`
Social	`post/[id]` · `story/[id]` · `user/[id]` · `profile/[id]` · `search` · `messages/` · `notifications`
Learning	`course/[id]` · `lesson/[id]` · `certificate/[id]` · `certificates` · `media`
Live	`live/[roomId]` · `recordings` · `tutoring/` · `tutoring/[id]` · `bookings`
Projects	`projects/` · `projects/create` · `projects/[id]` · `projects/showcase` · `projects/certificate/[id]`
Creators	`creator/` · `creator/courses/` · `creator/courses/create` · `creator/payouts` · `creator/tutoring` · `creator/events` · `creator/media` · `creator/analytics` · `creator/organizations/create`
Organizations	`organizations` · `organization/[id]` · `organization-registration`
Other	`admin` · `feedback` · `premium` · `privacy` · `terms`

Shared logic · `src/lib`

- `supabase` client · `nesba` general data helpers
- `payments` · `tutoring` · `projects` · `recordings`
- `lesson-activities` (quizzes, assignments)
- `upload` (progress, 50 MB limit) · `story-filters` (18 filters)
- `push` · `calendar` · `location` · `theme-preference`

Shared UI · `src/components`

- Course cards, badges, avatars, legal page and links
- Lesson quiz and assignment panels, discount-code manager
- Live: 3D lecture hall, live chat
- Emoji picker (full on web, simple grid on phones)

Theme

Light by default, optional dark mode. One colour palette in `src/constants/theme.js` ; the choice is saved on the device and read at start-up.

Web vs phone files

Where web and phone differ, a file comes in two versions: `name.web.js` and `name.native.js` (push, calendar, theme storage, emoji picker, 3D hall).

5 · Backend: Supabase services

Supabase is NESBA's main backend: database, accounts, files, live updates and server code in one managed service (region: EU · Frankfurt).

Postgres database

About 45 tables. Every table has **row-level security**: rules in the database decide which rows each signed-in user may read or change, so the app cannot leak data even if a screen has a bug. Sensitive actions (grading, payments, moderation) run as checked database functions.

Auth

Accounts with email and password, or Google / Apple. Each account has a `profiles` row with name, photo, role (student, teacher, mentor, organization, admin) and onboarding status. A trigger stops users from changing their own role or verification.

Realtime

The app subscribes to new messages, notifications and live-room changes, so pop-ups, unread counts and the live room update instantly.

Migrations

Every database change is a numbered SQL file in `src/sql` (001 → 034), run in order in the Supabase SQL editor. This is the history of the database.

Storage buckets

Bucket	Public?	Holds	Who can upload
`avatars`	public	Profile photos	Owner, in their own folder
`posts`	public	Post photos/videos, comment images	Owner, in their own folder
`stories`	public	Story media and songs (deleted nightly after expiry)	Owner, in their own folder
`lesson-files`	public	Lesson videos, audio, documents	Course teacher
`message-media`	private	Direct-message attachments (signed links)	Signed-in users
`nesba-content`	private	Protected course content	Creators
`recordings`	private	Live session recordings (2-hour signed links)	LiveKit, via server function
`public assets`	public	Shared images	Admins

6 · Backend: server functions and scheduled jobs

Code that must not run on a user's device (secret keys, payments, moderation) runs as Supabase Edge Functions.

Function	Called by	What it does
`livekit-token`	App (live room)	Checks the user may join, then issues a LiveKit access token with the right permissions (listener or speaker).
`livekit-participants`	App	Lists who is in a live room.
`livekit-moderate`	Host	Mute, remove, invite to speak, move to audience; enforced both in the database and in LiveKit.
`livekit-record`	Host / viewers	Starts and stops a room recording (uploaded to the `recordings` bucket) and creates watch links.
`stripe-connect`	Creators	Creates the creator's Stripe Express account, onboarding and dashboard links, payout status.
`stripe-checkout`	Learners	Prices from the database (never from the app), applies discount codes, records a pending purchase or booking, opens Stripe Checkout with the 20% platform fee.
`stripe-webhook`	Stripe	The only place a payment counts as done: enrolls the buyer or confirms the booking and creates the private live event. Verified by Stripe's signature.
`send-push`	Database trigger	Sends each new notification to the user's phones through Expo Push, respecting their settings.
`cleanup-stories`	pg_cron, nightly	Deletes expired stories and their files, and leftover files from failed uploads.

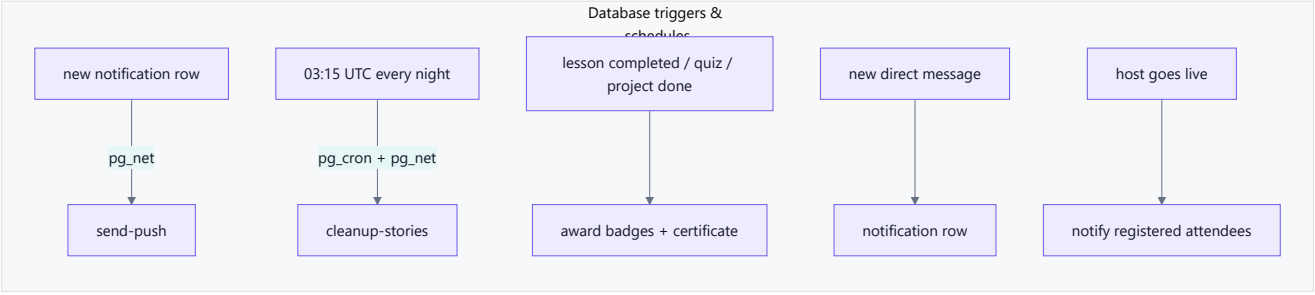


Figure 4 · Automatic work done by the database itself.

7 · Data model

The main tables, grouped by area. Every table belongs to a user through `profiles`.

profiles · every table belongs to a user

Learning

courses → course_sections → course_modules (lessons)

lessons → lesson_content · quiz_questions (+ answer keys) · lesson_assignments (+ submissions)

enrollments · lesson_progress → certificates · user_badges

Social

posts → post_comments · post_likes

stories → story_views

direct_messages · follows

notifications → push_tokens

Live

events → event_registrations

live_audio_rooms → participants · live chat

live_recordings

Payments

creator_payout_accounts

course_purchases · coupons

tutoring_offers → tutoring_bookings → tutoring_reviews

Groups

organizations → organization_members

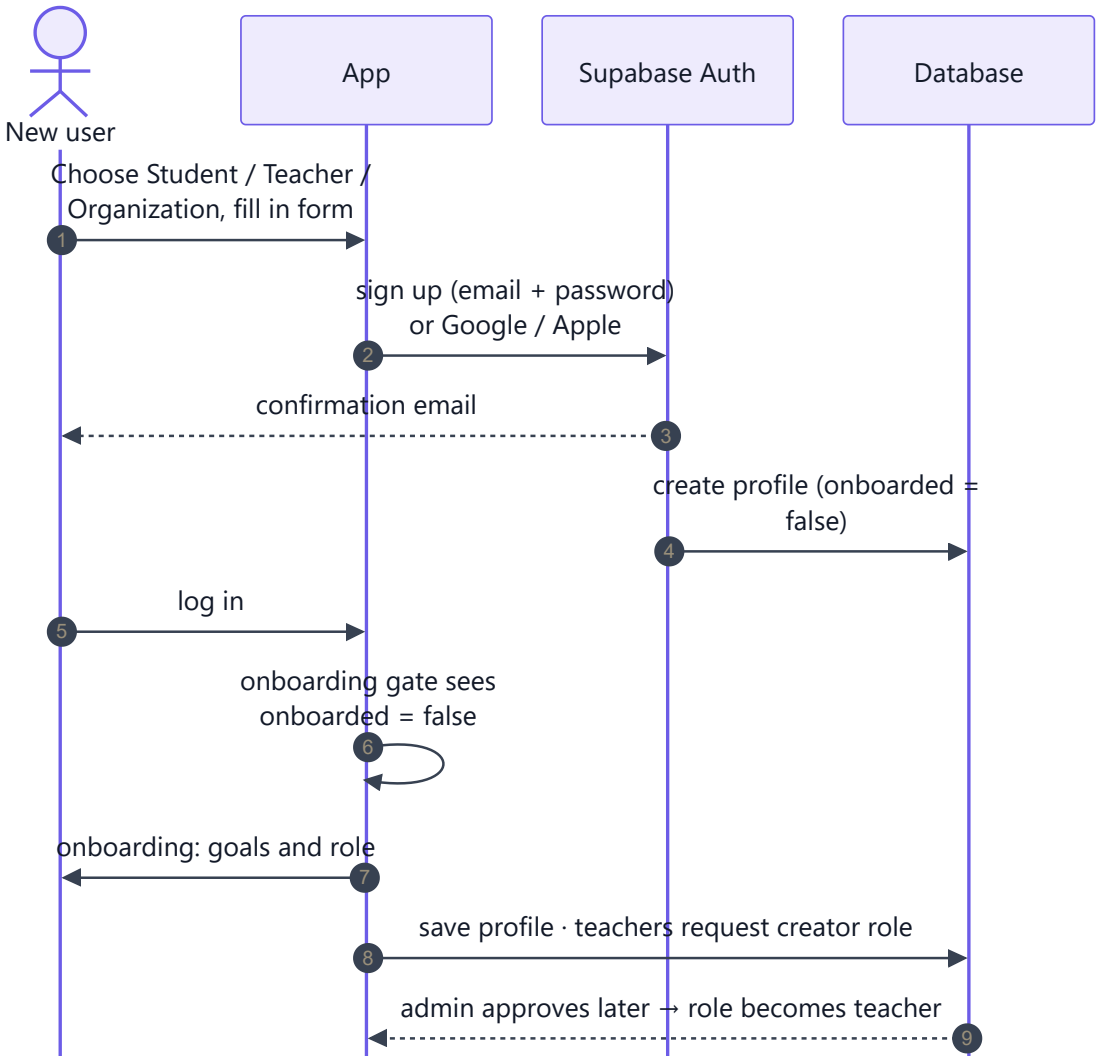
project_groups → members · posts · tasks

8 · Users, roles and permissions

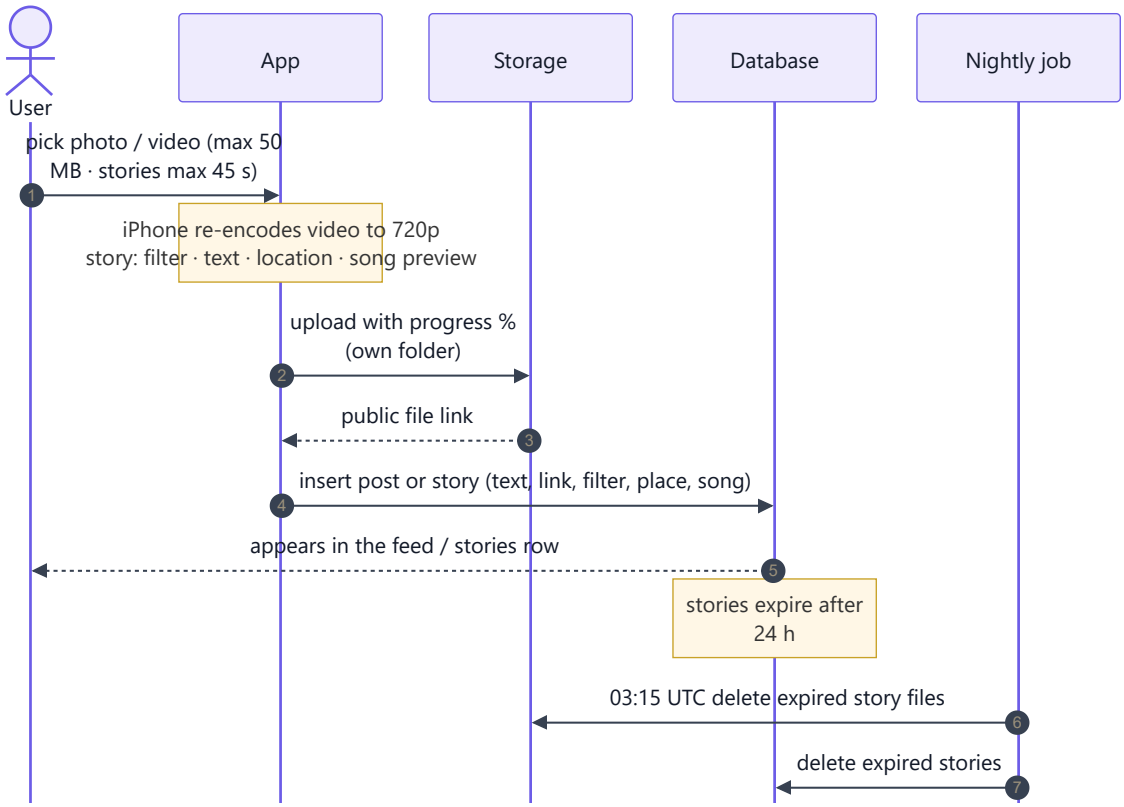
Can…	Guest	Student	Teacher / Mentor	Organization	Admin
Browse courses, posts, projects, legal pages	✓	✓	✓	✓	✓
Post, comment, stories, messages, follow	–	✓	✓	✓	✓
Enroll, learn, quizzes, assignments, certificates	–	✓	✓	✓	✓
Join live sessions, book private sessions	–	✓	✓	✓	✓
Start or join practical projects	–	✓	✓	✓	✓
Create courses, events, media; host live rooms	–	–	✓ after approval	✓ after approval	✓
Sell courses & private sessions, discount codes	–	–	✓ with Stripe	✓ with Stripe	✓
Approve creators and organizations, platform metrics	–	–	–	–	✓

> Teachers and organizations sign up like everyone else and request their role. An admin approves it. Users can never give themselves a role: the database blocks it.

9 · Flow: sign-up, login and onboarding



10 · Flow: posts, stories and uploads



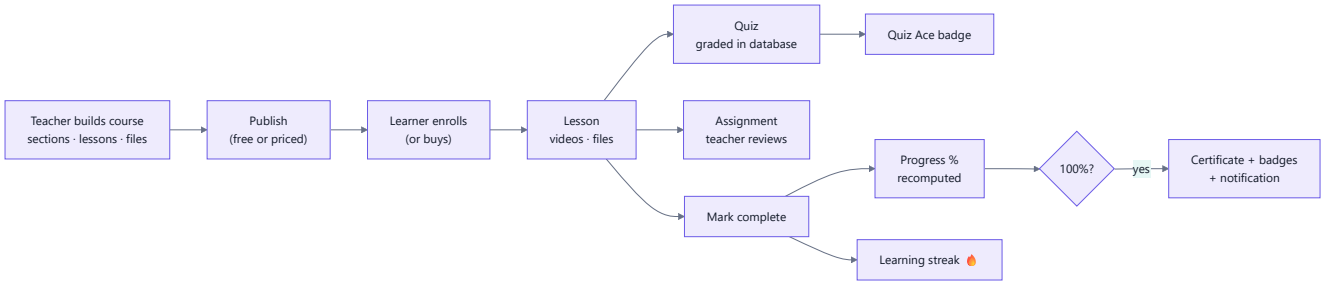
Stories

- 18 Instagram-style filters, shown the same in editor and viewer
- "Use my current location" (phone GPS or browser, place name via OpenStreetMap)
- Song plays while editing; only the first 45 seconds play
- Views recorded once per viewer

Feed & social

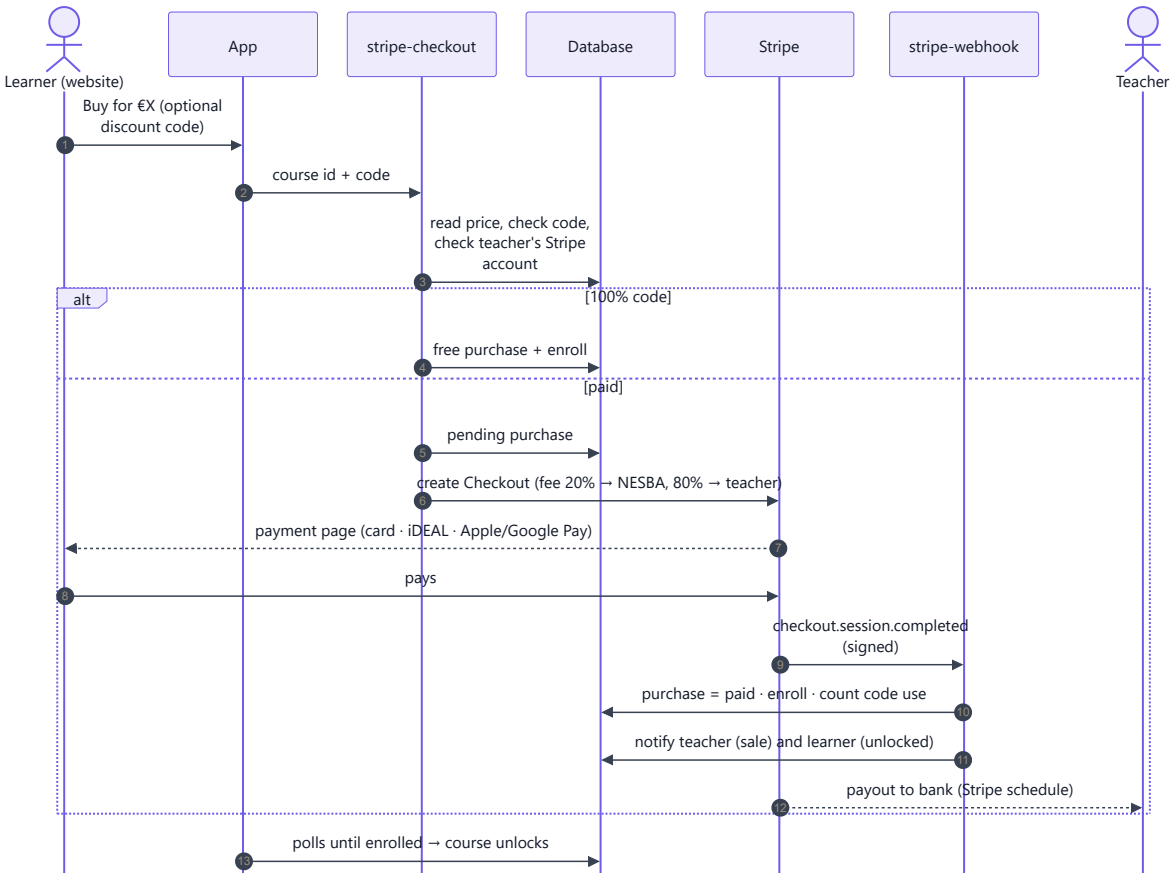
- Posts with text, photo, video or link; likes; threaded comments with images
- @mentions create notifications
- Follows, profiles, search for people
- Direct messages with attachments and realtime delivery

11 · Flow: courses, lessons, quizzes and certificates



> Quiz answers are stored in a separate table that learners cannot read. The database grades the answers and only then reveals the correct ones.

12 · Flow: buying a premium course



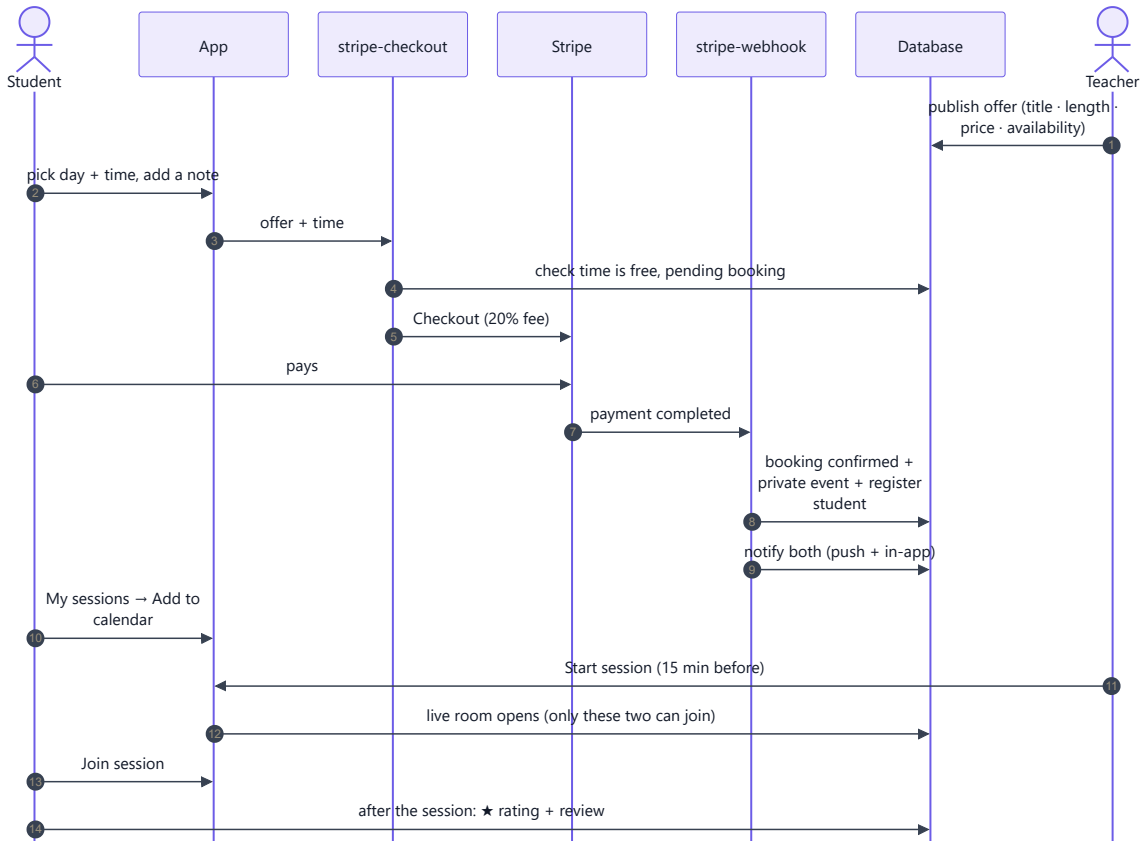
Creator setup

- Creator Studio → Payouts → Connect Stripe (Express, NL)
- Prices can only be set once Stripe can pay the creator
- Discount codes: % off, one or all courses, expiry, max uses

Safety rules

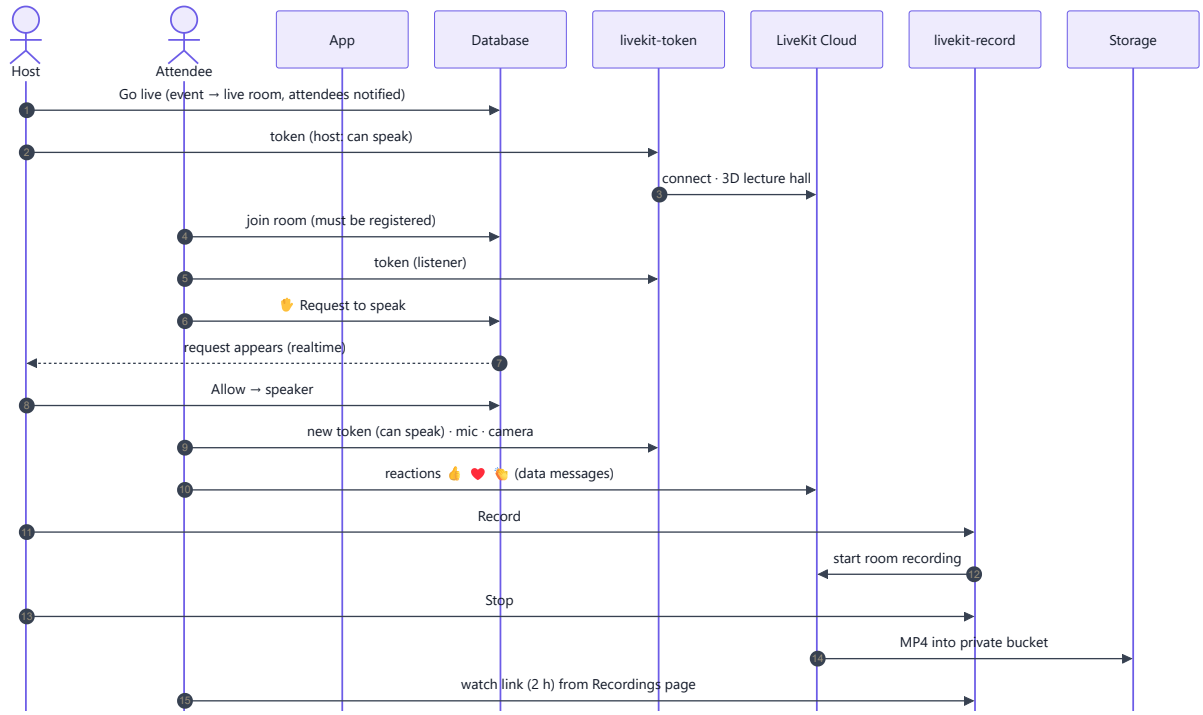
- Learners cannot enroll in a paid course without a paid purchase (database guard)
- Only the webhook can mark a purchase as paid
- Phone apps send buyers to the website (no app-store fees)

13 · Flow: booking a private session



> Private sessions reuse the live room. The event is marked **private**, so it is invisible to everyone except the teacher and the student who paid.

14 · Flow: live sessions and recordings



Everyone

Audio, 3D hall, chat, reactions, request to speak, view shared screen.

Speakers

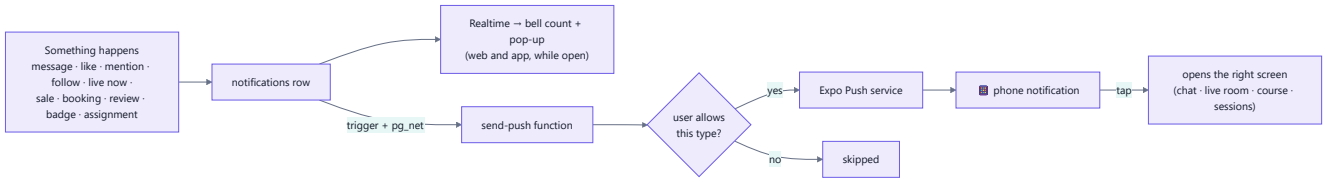
Microphone, camera tiles, screen sharing.

Host

Allow/decline requests, mute, mute all, move to audience, remove, record, end session.

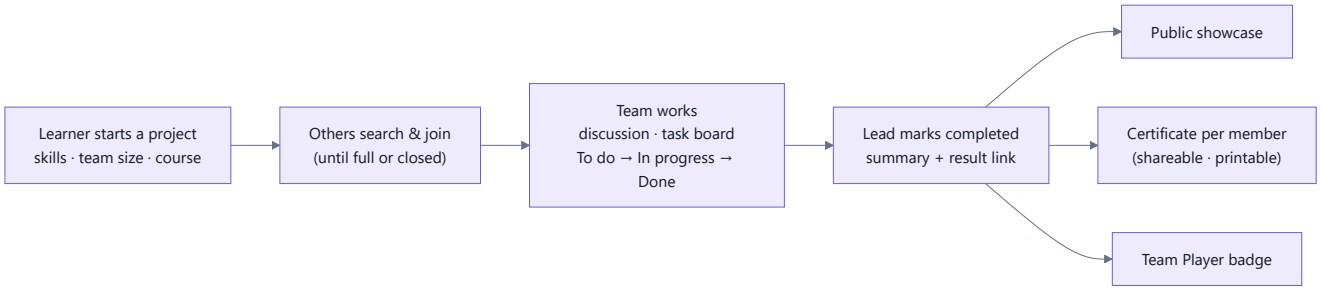
Live rooms currently run in the web version; the phone apps show a link to the website. Native live rooms need LiveKit's React Native SDK and a custom app build.

15 · Flow: notifications and push



Each phone registers a push token when someone signs in, and removes it when they sign out. Tokens from uninstalled apps are deleted automatically. Push needs the phone app (not the website) and an Expo project ID (`eas init`).

16 · Flow: practical projects



17 · Security model

Data access

- Row-level security on every table: users only see and change what the rules allow
- Privileged changes (roles, grading, payments, moderation) only through checked database functions
- Internal functions are not callable from the API (checked by Supabase's security advisor)

Accounts

- Passwords handled by Supabase Auth (never stored by NESBA)
- No public "does this email exist?" lookup
- Users cannot raise their own role or verification
- to do turn on leaked-password protection

Payments

- Card and bank details go straight to Stripe; NESBA never sees them
- Prices come from the database, not from the app
- Webhook requests must carry Stripe's signature
- Return links only to NESBA's own website

Content & files

- Uploads only into the user's own folder
- Private files (messages, recordings) via short-lived signed links
- Locked lessons and paid courses blocked in the database, not only in the app
- Production builds hide debug logs

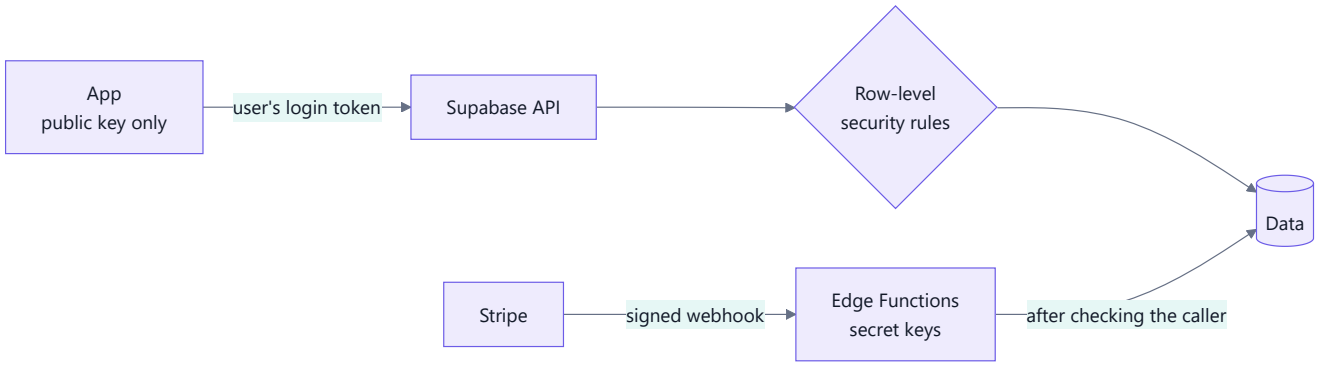


Figure · Two doors into the data: the app (limited by rules) and server functions (which check the caller themselves).

18 · Configuration, deployment and operations

Part	Where it lives	How it is updated
Source code	GitHub · `aelomerabi-oss/NESBA`	Branches + pull requests; current work on `claude/features-round-2`.
Website	Vercel or Netlify (to choose)	`npm run build:web` → `dist`; routing files already included.
iPhone / Android apps	Expo EAS → App Store / Google Play	`eas build` and `eas submit`; needs Apple ($99/yr) and Google ($25 once) accounts.
Database	Supabase project `xrgyqsfnaquypahiyqzb`	Run the next `src/sql/0NN_*.sql` file in the SQL editor.
Server functions	`supabase/functions/*`	Deploy with the Supabase CLI or tooling; settings in `supabase/config.toml`.
Secrets	Supabase → Edge Functions → Secrets	Changed in the dashboard; never committed to GitHub.
Payments	Stripe dashboard (sandbox now)	Webhook endpoint → `stripe-webhook`; switch to live keys at launch.
Live video	LiveKit Cloud	URL and keys as Supabase secrets.

Secrets (in Supabase, never in the app)

`STRIPE_SECRET_KEY` · `STRIPE_WEBHOOK_SECRET` · `SITE_URL` · `LIVEKIT_URL` · `LIVEKIT_API_KEY` · `LIVEKIT_API_SECRET` · `RECORDINGS_S3_ACCESS_KEY` · `RECORDINGS_S3_SECRET_KEY`. The app only contains the public Supabase key.

Developer tools

Checks

- `npm run shots`: opens pages in Chrome, saves screenshots, reports errors
- ESLint for code quality
- Supabase security advisor after every database change

Local development

- `npx expo start`: web at `localhost:8081`, phones via Expo Go
- `npx supabase` CLI for functions and secrets
- Later: Docker for a local test database and Strix security scans

19 · Capacity and costs (year 1, ~79 users)

Resource	Estimated need	Supabase Free	Supabase Pro (~$25/mo)
Database	50–200 MB	500 MB	8 GB
Files (posts, lessons, recordings)	45–100 GB	1 GB	100 GB + ~$0.02/GB
Largest single file	recordings 300–700 MB/hour	50 MB	configurable
Video downloads / month	50–150 GB	5 GB	250 GB
Backups · no pausing	needed with real users	no	daily backups

LiveKit

Audio sessions for this group size fit the lowest tiers; cameras and recording cost more per minute.

Stripe

No monthly fee: a fee per payment plus a small monthly fee per paid creator account.

Hosting

Vercel Pro (~$20/mo) or Netlify; the free Vercel plan does not allow commercial use.

> Before go-live: upgrade Supabase to **Pro**. If recordings grow large, move them to Cloudflare R2 or a video service (Mux / Cloudflare Stream). No Docker or Kubernetes is needed.

20 · Go-live checklist

1. Register the company; activate the Stripe account (verify business, bank).
2. Stripe live mode: new webhook (same URL, 5 events); replace `STRIPE_SECRET_KEY` and `STRIPE_WEBHOOK_SECRET`.
3. Choose hosting and domain; set `SITE_URL`; add the domain to Supabase Auth URLs.
4. Upgrade Supabase to Pro; turn on leaked-password protection.
5. Fill in the contact email on the legal pages; have Privacy and Terms reviewed.
6. Recordings: S3 access keys in Supabase; enable recording in LiveKit.
7. Phone apps: `eas init`, test builds, App Store / Google Play listings.
8. Install Docker; local test database; Strix security scan.
9. End-to-end tests: sign-up, course purchase + refund, private session, live room, push.
10. Closed beta with a small group; collect feedback; then open to the public.